Deadlock Analysis and Prevention

Name: _____

ID: _____

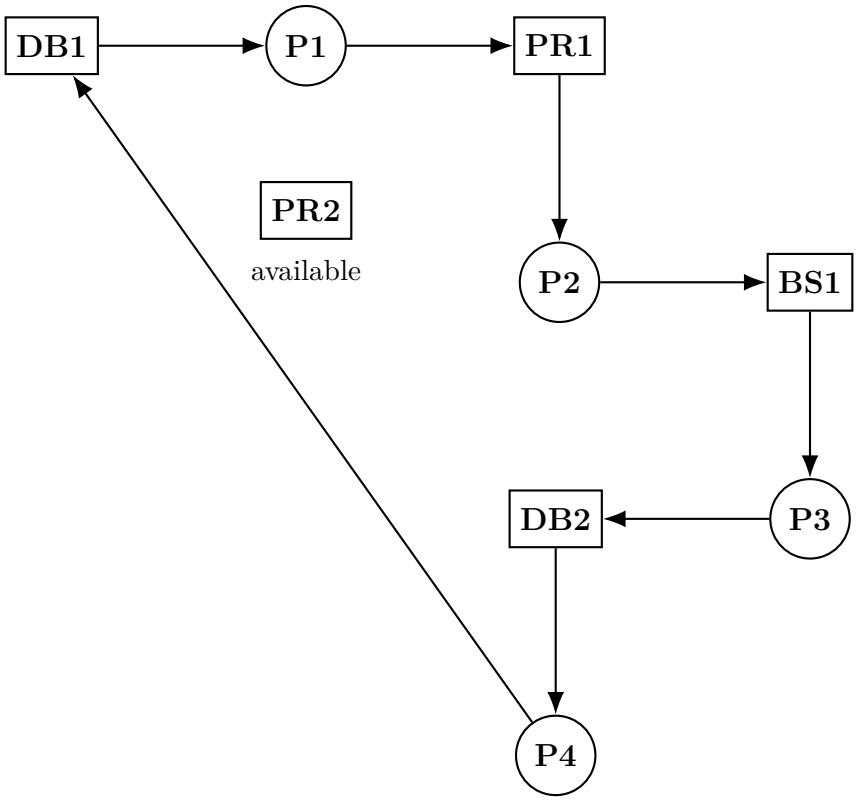
Course: Operating Systems

Given System State

Process	Allocated Resource	Waiting For
P1	DB1	PR1
P2	PR1	BS1
P3	BS1	DB2
P4	DB2	DB1

Available resources: PR2 only. The analysis treats DB1, DB2, PR1, PR2, and BS1 as exact resource instances, because the question specifies exact resources in the waiting column.

1. Resource Allocation Graph



The cycle is:

$$\mathbf{P1} \rightarrow \mathbf{PR1} \rightarrow \mathbf{P2} \rightarrow \mathbf{BS1} \rightarrow \mathbf{P3} \rightarrow \mathbf{DB2} \rightarrow \mathbf{P4} \rightarrow \mathbf{DB1} \rightarrow \mathbf{P1}$$

2. Allocation, Request, and Available Vectors

Allocation	DB1	DB2	PR1	PR2	BS1
P1	1	0	0	0	0
P2	0	0	1	0	0
P3	0	0	0	0	1
P4	0	1	0	0	0

Request	DB1	DB2	PR1	PR2	BS1
P1	0	0	1	0	0
P2	0	0	0	0	1
P3	0	1	0	0	0
P4	1	0	0	0	0

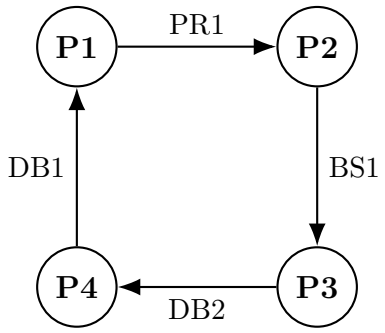
Available DB1	DB2	PR1	PR2	BS1
0	0	0	1	0

Only PR2 is available, but no process is waiting for PR2. Therefore no process can be chosen as the first process in a safe sequence.

3. Deadlock Analysis

The waiting dependency is:

P1 waits for P2, P2 waits for P3, P3 waits for P4, and P4 waits for P1.



This wait-for graph has the cycle P1 → P2 → P3 → P4 → P1. Since no process releases its allocated resource while waiting, none of them can complete. Hence, the system is **deadlocked**.

4. Four Necessary Conditions

Condition	How it appears in this case
Mutual exclusion	DB1, DB2, PR1, PR2, and BS1 are limited resources assigned to one process at a time.
Hold and wait	Each process holds one resource while waiting for another resource.
No preemption	The problem states that no process releases its allocated resource while waiting.
Circular wait	P1 waits for P2, P2 waits for P3, P3 waits for P4, and P4 waits for P1.

All four conditions hold simultaneously, so deadlock is confirmed.

5. Prevention and Avoidance

Method	Use	Limitation
Request all resources at once	Prevents hold-and-wait.	Low resource utilization.
Resource ordering	Prevents circular wait by forcing a fixed request order.	All programs must follow the same rule.
Preemption and rollback	Recovers resources from blocked processes.	Hard for printer, database, and backup operations.
Banker's algorithm	Grants a request only if the state remains safe.	Needs maximum demand information in advance.
Timeout and retry	Detects long waits and restarts affected processes.	Recovery technique, not full prevention.

6. Recommendation

The best practical solution is **resource ordering**, supported by timeout-based rollback. A suitable global order is:

Database connections → Printers → Backup storage

Each process must request resources only in this order. This directly prevents circular wait, which is the main cause of the observed deadlock. Timeout and rollback should also be used so that abnormal or long-waiting processes release resources and do not block the server indefinitely.

Conclusion

The system is deadlocked because the resource allocation graph and wait-for graph both contain a cycle, and no process can start a safe completion sequence. The most suitable solution is to enforce a fixed resource-ordering policy with timeout-based recovery.